

TP 03 — Catalogue de films avec Custom Hooks et TanStack Query

Énoncé apprenant — React • Custom Hooks • TanStack Query • Debounce • localStorage

Objectif

Créer une petite application React permettant de consulter un catalogue de films en utilisant TanStack Query pour les appels API, des custom hooks pour factoriser la logique, un hook de debounce pour la recherche et le localStorage pour gérer les favoris.

Fonctionnalités attendues

- afficher une liste de films populaires au chargement
- rechercher des films à partir d'un champ texte
- déclencher la recherche après un court délai
- changer de page dans les résultats
- ajouter ou retirer un film des favoris
- conserver les favoris après rechargement de la page

Contraintes

- utiliser React
- utiliser TanStack Query
- créer au minimum les hooks usePopularMovies, useSearchMovies, useDebounce et useLocalStorage
- ne pas effectuer tous les appels API directement dans les composants
- garder des composants centrés sur l'affichage

API utilisée

Vous pouvez utiliser l'API de The Movie Database (TMDB).

Créer un fichier .env.local à la racine du projet :

```
VITE_TMDB_API_KEY=votre_cle_api  
VITE_TMDB_BASE_URL=https://api.themoviedb.org/3
```

Travail demandé

1. Installer TanStack Query

- Installer la bibliothèque nécessaire avec `npm install @tanstack/react-query`.

2. Configurer TanStack Query dans l'application

- Mettre en place QueryClient et QueryClientProvider dans le point d'entrée de l'application.
- L'ensemble de l'application doit être enveloppé dans le provider.

3. Créer un fichier d'accès API

- Créer le fichier src/api/movies.js.
- Dans ce fichier, créer les fonctions getPopularMovies(page) et searchMovies(query, page).
- Ces fonctions doivent appeler l'API TMDb et retourner les données JSON.

4. Créer un custom hook pour les films populaires

- Créer le fichier src/hooks/useMovies.js.
- Créer un hook usePopularMovies(page).
- Ce hook doit utiliser useQuery, récupérer les films populaires, gérer la page courante, utiliser une queryKey claire et conserver les anciennes données pendant le chargement d'une nouvelle page.

5. Créer un custom hook pour la recherche

- Dans le même fichier, créer un hook useSearchMovies(query, page).
- Ce hook doit utiliser useQuery, lancer une recherche à partir de 2 caractères minimum, dépendre du texte recherché et du numéro de page, et utiliser une queryKey adaptée.

6. Créer un custom hook useDebounce

- Créer le fichier src/hooks/useDebounce.js.
- Créer un hook useDebounce(value, delay) qui prend une valeur en entrée, retourne une version retardée de cette valeur et attend un délai avant de propager la nouvelle valeur.

7. Créer un custom hook useLocalStorage

- Créer le fichier src/hooks/useLocalStorage.js.
- Créer un hook qui fonctionne comme un useState mais avec persistance dans le localStorage.
- Ce hook servira à stocker les identifiants des films favoris.

8. Créer le composant MovieCard

- Créer le fichier src/components/MovieCard.jsx.
- Ce composant doit afficher pour chaque film : l'affiche, le titre, la note, l'année de sortie et un bouton pour ajouter ou retirer le film des favoris.

9. Créer l'écran principal

- Dans App.jsx, construire l'interface principale avec un titre, un champ de recherche, une zone d'affichage des résultats, une pagination et une gestion des favoris.

- Comportement attendu : si le champ de recherche est vide, afficher les films populaires ; sinon afficher les résultats de recherche ; utiliser la valeur debouncée ; remettre la page à 1 quand la recherche change ; afficher un message pendant le chargement ; afficher un message en cas d'erreur.

Structure suggérée

```
src/
  api/
  movies.js
  components/
  MovieCard.jsx
  hooks/
  useMovies.js
  useDebounce.js
  useLocalStorage.js
  App.jsx
  main.jsx
```

Interface minimale attendue

Zone de recherche

Un champ texte permettant de rechercher un film.

Liste des films

Une grille ou une liste de cartes affichant les films retournés.

Pagination

Deux boutons minimum : précédent et suivant. La pagination doit mettre à jour les résultats affichés.

Favoris

Un film ajouté en favori doit être visuellement identifiable.

Données utiles d'un film

- id
- title
- poster_path
- vote_average
- release_date

Pour l'image : `https://image.tmdb.org/t/p/w300`

Exemple : `const imageUrl = `https://image.tmdb.org/t/p/w300${movie.poster_path}`;`

Fonctionnalités obligatoires

- affichage des films populaires
- recherche avec debounce
- pagination
- favoris sauvegardés dans le localStorage
- séparation logique / affichage avec hooks personnalisés

Bonus

- afficher le nombre de favoris

- ajouter un filtre « favoris uniquement »
- désactiver le bouton « précédent » sur la première page
- afficher une image par défaut si poster_path est absent
- styliser proprement la carte film

Critères d'évaluation

Organisation du code	Appels API sortis des composants, hooks bien nommés, composants lisibles.
Utilisation des hooks	Bon usage de useQuery, des custom hooks, du debounce et de la persistance.
Comportement	Chargement correct, erreurs gérées, recherche fonctionnelle, pagination fonctionnelle et favoris persistants.

Consigne finale

L'objectif n'est pas seulement d'afficher des films. L'objectif est surtout de montrer que vous savez isoler une logique réutilisable dans des custom hooks, utiliser TanStack Query pour gérer les données serveur et garder des composants React propres et lisibles.